

Manual STP Suitability

Technical Documentation

Architecture | User Flow Flowchart | API Reference | Store Design

DSS | STP | Wastewater Treatment Planning

v1.0 | June 2026

SLCR Deployment -- Frontend Manual Mode Module

Table of Contents

1. Module Overview	pg 3
2. Architecture Diagram	pg 4
3. User Flow Flowchart	pg 5
4. Complete User Flow -- Step by Step	pg 6
5. Store Architecture & State Management	pg 7
6. API Service Reference	pg 8
7. TypeScript Types & Interfaces	pg 9
8. Map Layers & Styling	pg 10
9. STP Technology Scoring Algorithm	pg 11
10. Multi-Polygon Flow	pg 12
11. Key Algorithms & Logic	pg 13
12. Configuration & Constants	pg 14
13. Error Handling & Edge Cases	pg 15

1. Module Overview

The **Manual STP Suitability** module is a geospatial decision-support tool for Sewage Treatment Plant site selection. Planners delineate a study area (shapefile, KML, or drawn polygon), run weighted suitability analysis, score candidate STP technologies, and find optimal clusters with road-network distance calculations to drain points.

1.1 Module Location

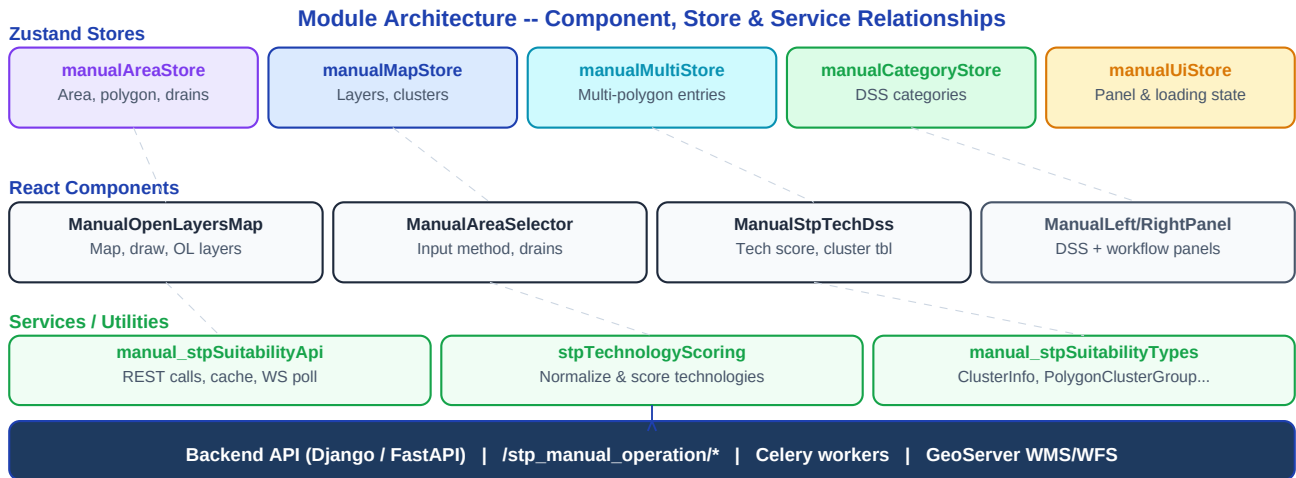
Root Path	frontend/app/dss/stp/wwt/stp_suitability_v2/manual/
Route	/dss/stp/wwt/stp_suitability_v2/manual
Framework	Next.js 14 (App Router) + React 18
Map Library	OpenLayers 9
State Management	Zustand (5 dedicated stores)
Styling	Tailwind CSS
Backend	Django REST + FastAPI + Celery + GeoServer

1.2 Key Capabilities

- **Three input methods:** Shapefile upload, KML upload, or interactive polygon drawing.
- **Multi-file / multi-polygon:** Process multiple regions simultaneously; each gets independent drain selection and cluster analysis.
- **Constraint checking:** Backend validates the area against constraint layers before proceeding.
- **DSS workflow:** Optional suitability raster from weighted condition + constraint layers.
- **Technology scoring:** Normalized multi-criteria scoring across 12 STP technologies.
- **Cluster analysis:** Finds top-N clusters within the suitability raster; computes road-network distances to drains.
- **Cluster area warning:** Alerts if no cluster has sufficient land for the required STP footprint.
- **DPR cost estimation:** Optional project cost report from selected technology + cluster.

2. Architecture Diagram

Relationships between Zustand stores, React components, service utilities, and the backend API.



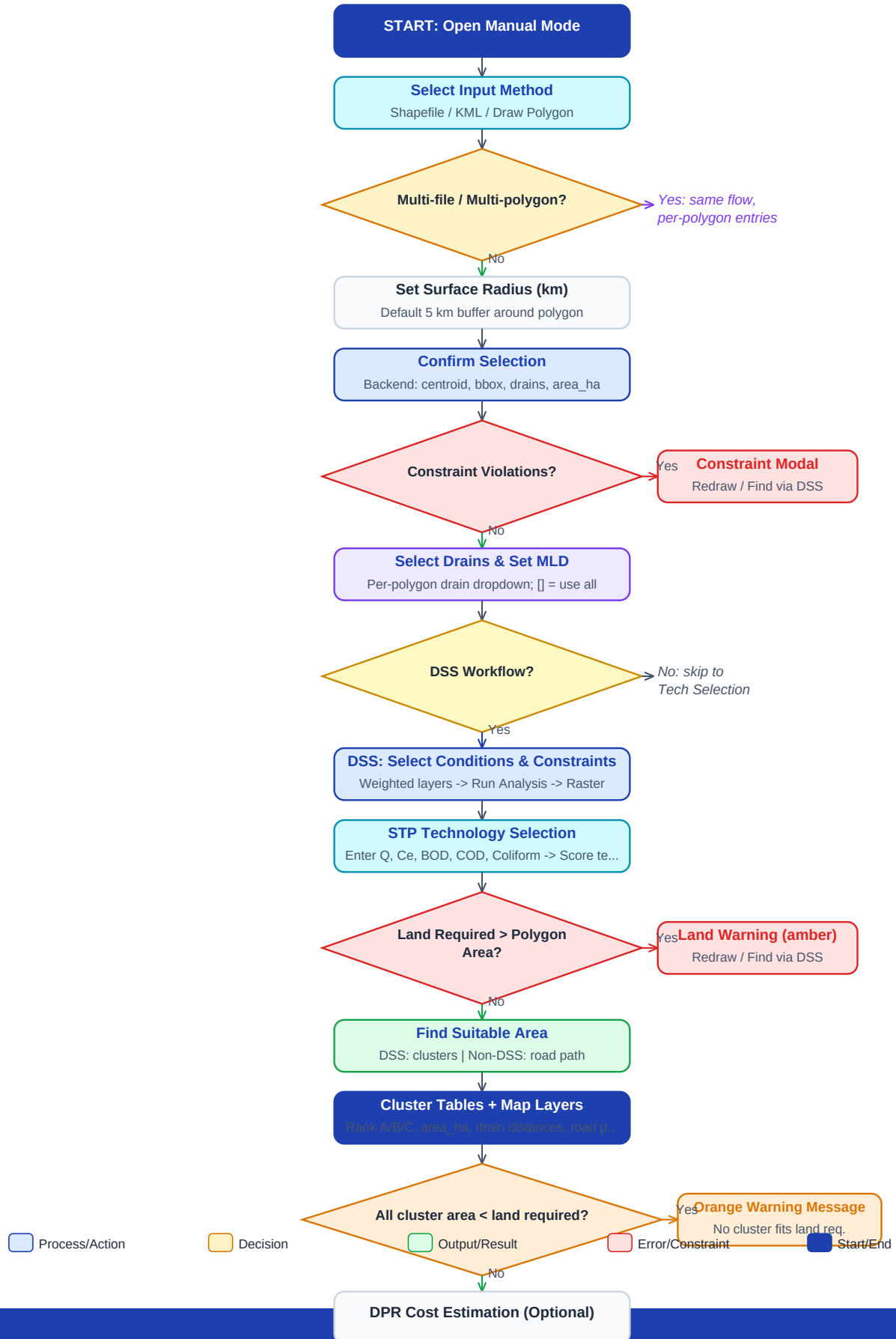
2.1 Component Responsibilities

Component	Responsibility	Key Stores
ManualOpenLayersMap	OL map, draw/modify/snap interactions, all GeoServer WMS/WFS layers, admin layer toggles	mapStore, areaStore, multiStore
ManualAreaSelector	Left panel: method selector, file uploader, drain picker, surface radius, confirm button	areaStore, mapStore, multiStore
ManualStpTechnologyDss	Right panel: STP parameters, technology ranking table, cluster tables, land/cluster warnings, DPR button	mapStore, categoryStore
ManualLeftPanel	Shell wrapping ManualAreaSelector in the left drawer	uiStore
ManualRightPanel	Orchestrates DSS workflow panels (category slider -> suitability -> tech selection)	uiStore, categoryStore
MultiDrainSelector	Per-polygon drain dropdown accordion for multi-polygon flow	multiStore, mapStore
ClusterDistancesTable	Single-polygon cluster table: distance legend, drain chips, info tooltip, road-path toggle	mapStore
MultiClusterDistancesTable	One ClusterTableShell per polygon group (multi-polygon flow)	mapStore

3. User Flow Flowchart

Complete decision tree from opening the manual mode to cluster results and DPR estimation.

Manual STP Suitability -- Complete User Flow



4. Complete User Flow -- Step by Step

Phase 1 -- Area Selection

Step 1: Select Method	Choose Shapefile, KML, or Draw Polygon from the left panel radio group.
Step 2: Upload / Draw	Shapefile/KML: upload files, click Upload, preview appears on map. Polygon: toggle Draw on Map, click vertices, double-click to close. Multiple polygons can be drawn sequentially.
Step 3: Preview	Polygon outline appears in cyan dashed style before confirmation.
Step 4: Surface Radius	Adjust buffer zone (default 5 km) controlling drain search area around the polygon.
Step 5: Confirm Selection	POST to /stp_manual_operation/confirm_manual_area. Returns: village vector layer, polygon layer, centroid, bbox, area_ha, raster layers.

Phase 2 -- Constraint Checking

After confirmation, the backend checks constraint layers within the polygon. If violations exist, a modal appears with two options:

- **Redraw Area** -- unlocks, clears map, returns to Phase 1.
- **Find through DSS** -- proceeds to DSS suitability workflow (Phase 4).

Phase 3 -- Drain Selection & MLD Input

Drain points in the buffer bbox are fetched and shown in a dropdown. User can leave All Drains checked (default = all used), uncheck individually, or uncheck All to clear. User also enters **Drain Capacity (MLD)** which drives technology scoring and land calculation: **Land Required (ha) = land_per_MLD x Q (MLD)**.

Phase 4 -- DSS Suitability Analysis (Optional)

User selects weighted condition layers (slope, land use etc.) and constraint layers. Clicking Analyze runs suitability raster generation. The map shows this raster; a village-level risk score table appears in the right panel.

Phase 5 -- STP Technology Selection

Q (MLD)	STP design capacity; drives land and cost calculations.
Ce (Rs/kWh)	Electricity cost; affects O&M; score.
BOD / COD (mg/L)	Influent concentrations; determine effluent compatibility score.
Coliform (MPN/100mL)	Faecal coliform load; same.
Land Required	= land_per_MLD x Q. Shown as constraint in Phase 6.

Phase 6 -- Land Violation Check

If **Land Required > Marked Area (ha)**, an amber warning banner replaces the Find Suitable Area button. Options: Redraw Area or Find through DSS.

Phase 7 -- Find Suitable Area

DSS Mode	POST /find_area_cluster. Backend clusters suitability raster, ranks by proximity to centroid, returns top-N clusters (A/B/C) with area_ha, dist_to_polygon_m, drain distances, pre-computed road-path layers.
Non-DSS Mode	POST /find_manual_path. Finds shortest road path from centroid to drains. Returns path layer and cluster distances.
Multi-Polygon	Calls findMultiArea / findMultiPath sequentially per polygon to avoid Celery worker saturation.

Phase 8 -- Cluster Results & Area Warning

Cluster tables render showing rank (A/B/C), area_ha (green), distance from polygon, and drain road distances. Clicking a cluster row toggles its road-path layer on the map.

Cluster Area Warning: If every cluster's area_ha is less than landRequired, an orange warning appears below the last table: 'Land required (X.XX ha) is too large to match any cluster area. No cluster has sufficient land for the selected technology.'

5. Store Architecture & State Management

Five Zustand stores manage all module state.

5.x manualAreaStore

Field	Type	Description
selectedMethod	"shapefile" "polygon" "kml"	Active input method
uploadedFiles	File[]	Files queued for upload
drawnPolygon	{ geojson, label } null	Latest confirmed polygon
drawnPolygons	{ geojson, label }[]	All drawn polygons
selectionsLocked	boolean	Prevents edits after confirm
selectionVectorLayer	string null	GeoServer villages layer name
polygonLayer	string null	GeoServer polygon outline layer
areaCentroid	[lat, lon] null	Polygon centroid for API calls
bufferBbox	[minLon, minLat, maxLon, maxLat]	Bbox for drain query
drainPoints	DrainPoint[]	All drains within bbox
selectedDrainNos	number[]	[] = use all; [1,2] = specific
drainCapacityMld	number null	Design flow in MLD
markedAreaHa	number	Polygon area in hectares
surfaceRadius	number	Buffer radius in km (default 5)
previewGeojson	FeatureCollection null	Preview outline before confirm

5.x manualMapStore

Field	Type	Description
resultVectorLayer	string null	Cluster polygons GeoServer layer
resultPathVectorLayer	string null	Road path GeoServer layer
clusterDistances	ClusterInfo[] null	Single-polygon cluster results
multiClusterDistances	PolygonClusterGroup[] null	Multi-polygon cluster results
selectedRadioLayer	string null	Active raster for WMS display
rasterLayerInfo	ClipRasters null	Workspace + layer_name for WMS
layerOpacity	number	0-100 raster opacity
showLegend	boolean	Show/hide raster legend
drawingActive	boolean	OL draw interaction active

showDrainLabels	boolean	Drain No. text labels on map
showPolygonLabels	boolean	Polygon 1/2/3 labels on map
selectedClusterRank	number null	Which cluster road-path to show

5.x manualMultiStore

Field	Type	Description
polygonEntries	MultiPolygonEntry[]	Per-polygon: layers, centroid, bbox, area, drains
polygonResults	MultiPolygonResult[]	Per-polygon cluster + path results
drainCapacityMld	number null	Shared MLD across all polygons
selectionsLocked	boolean	Locks all polygon entries after confirm

5.x manualCategoryStore

Field	Type	Description
initialized	boolean	Categories fetched from backend
conditionCategories	Category[]	Available suitability layers
constraintCategories	Category[]	Available constraint layers
areaOptions	Stp_area[]	Technology land-per-MLD options
selectedCondition	SelectRasterLayer[]	User-selected conditions with weights
selectedConstraint	SelectRasterLayer[]	User-selected constraints with weights
tableData	DataRow[]	Suitability analysis CSV results
showTable	boolean	Show results table

5.x manualUiStore

Field	Type	Description
isRightPanelOpen	boolean	Right panel drawer state
showDssWorkflow	boolean	DSS path selected
treatmentLoading	boolean	Area-finding in progress
isPdfGenerating	boolean	DPR report generating
pendingTechnologyValues	TechnologyAreaSubmitValues null	Tech values pending after constraint

6. API Service Reference

All calls are in **manual_stpSuitabilityApi.ts**. Long-running tasks use Celery; frontend polls via WebSocket (HTTP fallback, 2s interval, 10 min timeout).

Method	Endpoint	Function	Description / I-O
POST	/confirm_manual_area	confirmManualAreaSelection	Accepts shapefile/KML/polygon. Returns vector layer, polygon layer, centroid, bbox, area_ha, raster layers. <i>In: method, files polygon kml, surface_radius</i> <i>Out: { vectorLayer, polygonLayer, centroidLat/Lon, bufferBbox, areaHa, rasterLayers }</i>
POST	/preview_polygon	previewPolygon	Returns GeoJSON FeatureCollection outline before confirmation. <i>In: files: File[]</i> <i>Out: GeoJSON.FeatureCollection</i>
POST	/confirm_multi_area	confirmMultiAreaSelection	Processes multiple files into separate polygon entries. <i>In: files, surface_radius</i> <i>Out: { results: MultiAreaConfirmSingleResult[] }</i>
POST	/confirm_multi_drawn_polygons	confirmMultiDrawnPolygons	Processes array of drawn GeoJSON polygons. <i>In: polygons: { geojson, label }[]</i> <i>Out: { results: MultiAreaConfirmSingleResult[] }</i>
POST	/stp_manual_check_constraints	checkManualConstraints	Checks polygon against constraint layers. Returns violations list. <i>In: polygon_geojson</i> <i>Out: { constraint_violations: string[], can_proceed: boolean }</i>
POST	/run_manual_suitability	runManualSuitabilityAnalysis	Weighted suitability analysis; generates raster on GeoServer. <i>In: data: SelectRasterLayer[], village_layer, method, file?, polygon?</i> <i>Out: { workspace, layer_name, csv_details: DataRow[] }</i>
GET	/fetch_drains_in_bbox	fetchDrainsInBbox	Drain points within bounding box. <i>In: bbox: [minLon, minLat, maxLon, maxLat]</i> <i>Out: { Drain_No, latitude, longitude }[]</i>

POST	/find_area_cluster	findSuitabilityAreaCluster	DSS mode: K-means clustering on suitability raster, ranks by proximity. Long-running (Celery). <i>In: treatment_technology, mld_capacity, layer_name, location, drain_points?, num_clusters?</i> <i>Out: { cluster_layer, cluster_distances: ClusterInfo[], suitable_path? }</i>
POST	/find_manual_path	findManualPath	Non-DSS mode: road-network path from centroid to drains. <i>In: location, drain_points, mld_capacity</i> <i>Out: { suitable_path, cluster_distances: ClusterInfo[] }</i>
POST	/find_multi_area	findMultiArea	Multi-polygon DSS: runs findSuitabilityAreaCluster sequentially per polygon. <i>In: entries: MultiPolygonEntry[], tech params</i> <i>Out: { results: MultiAreaSingleResult[] }</i>
POST	/find_multi_path	findMultiPath	Multi-polygon non-DSS: runs findManualPath sequentially per polygon. <i>In: entries: MultiPolygonEntry[], drain params</i> <i>Out: { results: MultiFindPathSingleResult[] }</i>
GET	/get_suitability_categories	fetchSuitabilityCategories	Available condition/constraint layers and technology land-per-MLD options. <i>In: --</i> <i>Out: { conditions, constraints, area_options: Stp_area[] }</i>

7. TypeScript Types & Interfaces

ClusterInfo

```
cluster_rank: number // 1=nearest
area_ha: number
dist_to_polygon_m: number
drains: { Drain_No: number; distance_m: number }[]
path_layer?: string | null // DSS mode only
```

PolygonClusterGroup

```
label: string // e.g. 'Polygon 1'
clusters: ClusterInfo[]
```

MultiPolygonEntry

```
index: number
vectorLayer: string // GeoServer villages
polygonLayer: string | null
centroid: [lat: number, lon: number]
bufferBbox: [minLon, minLat, maxLon, maxLat]
areaHa: number
drainPoints: DrainPoint[]
selectedDrainNos: number[] // [] = use all
displayRasters: ClipRasters[]
```

TechnologyAreaSubmitValues

```
landPerMld: number // ha/MLD
mldCapacity: number // Q in MLD
technologyName: string
numClusters?: number // 1-10, default 3
```

ManualAreaConfirmResult

```
rasterLayers: ClipRasters[]
vectorLayer: string | null
polygonLayer: string | null
centroidLat: number
centroidLon: number
bufferBbox: [minLon, minLat, maxLon, maxLat]
areaHa: number
```

StpTechnologyDssProps

```
canFindArea?: boolean
enableDprCostEstimator?: boolean
isFindingArea?: boolean
drainCapacityMld?: number | null
markedAreaHa?: number
showClusterSelect?: boolean
onRedrawPolygon?: () => void
onApplyDss?: (v: TechnologyAreaSubmitValues) => void
onFindArea?: (v: TechnologyAreaSubmitValues) => void | Promise
```

8. Map Layers & Styling

8.1 Layer Z-Index Stack

Z-Index	Layer	Type	Style
0	Base tile map (OSM)	TileLayer	Default OSM tiles
1	India boundary	VectorLayer (WFS)	Stroke #1e40af, no fill
13	Villages (confirmed area)	VectorLayer (WFS)	Indigo #6366f1, 10% fill
20	Draw layer (active drawing)	VectorLayer	Cyan #06b6d4 dashed 5,5
30	Suitability raster	ImageLayer (WMS)	Opacity: user-adjustable 0-100%
40	Result clusters (A/B/C)	VectorLayer (WFS)	Dark fill with rank text labels
41	Road path network	VectorLayer (WFS)	Green #16a34a, width 3
55	Drain points	VectorLayer (points)	Violet #7c3aed circles; optional No. labels
60	Polygon outline	VectorLayer	Cyan #06b6d4 dashed, 8% fill
70	Polygon labels (Polygon 1/2/3)	VectorLayer (text)	16px bold #4D5D53, toggle ON/OFF

8.2 Administrative Layer Toggles

India boundary	Toggles country boundary (WFS). Ref: multiPolygonOutlineLayersRef.
Drawn Polygon	Toggles uploaded/drawn polygon outline layers.
Confirmed Selection	Toggles village vector layers (per-polygon in multi mode).
Treatment Cluster	Toggles cluster result polygons (A/B/C). Ref: multiResultLayersRef.

9. STP Technology Scoring Algorithm

Implemented in `stpTechnologyScoring.ts`. Each technology is scored 0-10 across seven criteria; weighted sum determines the rank.

9.1 Scoring Criteria & Weights

Criterion	Weight Key	Calculation
Reliability	<code>C_WEIGHTS.rel</code>	Fixed value per technology (0-10)
Capital Cost	<code>C_WEIGHTS.cap</code>	Normalized: $10 \times (\text{max-cap})/(\text{max-min})$; higher = better
Land Requirement	<code>C_WEIGHTS.land</code>	Normalized: $10 \times (\text{max-land})/(\text{max-min})$; higher = better
O&M; Cost	<code>C_WEIGHTS.om</code>	Normalized over all technologies; higher = better
Ease of Operation	<code>C_WEIGHTS.ease</code>	Fixed value per technology (0-10)
Track Record	<code>C_WEIGHTS.track</code>	Fixed value per technology (0-10)
Effluent Compatibility	<code>C_WEIGHTS.effluent</code>	$\text{avg}(\text{BOD_score}, \text{COD_score}, \text{Coliform_score})$ from lookup

9.2 Land Required Formula

```
landRequired (ha) = selectedRawTech.land (ha/MLD) x params.Q (MLD)
```

Compared against `markedAreaHa` (polygon area check) and each `cluster.area_ha` (cluster area warning). Both produce distinct warnings.

10. Multi-Polygon Flow

When 2+ files are uploaded or 2+ polygons drawn, the module switches to multi-polygon mode. Each polygon is processed independently; drain capacity MLD is shared across all.

10.1 Entry Structure (MultiPolygonEntry)

index	Sequential identifier (0, 1, 2...)
vectorLayer	GeoServer WFS layer for this polygon's villages
polygonLayer	GeoServer polygon outline layer (may be null for drawn polygons)
centroid	[lat, lon] -- used as origin for road-path and cluster queries
bufferBbox	[minLon, minLat, maxLon, maxLat] -- for drain fetch
areaHa	Polygon area in hectares
drainPoints	All drain points within buffer bbox
selectedDrainNos	[] = use all drains; [1,2,3] = specific drain numbers
displayRasters	Raster layers generated for this polygon (STP_Suitability_P1, etc.)

10.2 Drain Selection Per Polygon

MultiDrainSelector renders a collapsible accordion with one PolygonDrainDropdown per entry. Each dropdown has its own uiSelected local state (null = all, [] = none, array = partial), completely decoupled from the store's selectedDrainNos semantics.

10.3 Sequential API Calls

Multi-polygon API calls are sequential (not parallel) to prevent Celery worker saturation. Each polygon's result is stored in polygonResults in multiStore.

10.4 Raster Layer Deduplication

Each polygon gets its own suitability raster (STP_Suitability_P1, P2...). The layer selector shows a single 'STP Suitability' entry; selecting it displays the first polygon's raster.

11. Key Algorithms & Logic

11.1 Drain Checkbox UX (uiSelected Pattern)

Two-state pattern distinguishes 'all selected' from 'none selected':

```
// uiSelected: null = all selected (visual), [] = none, [1,2] = partial
const [uiSelected, setUiSelected] = useState(null);
const isAllSelected = uiSelected === null;
const checkedIds = isAllSelected ? allIds : uiSelected;

const toggleAll = () => {
  if (isAllSelected) { setUiSelected([]); setSelectedDrainNos(allIds); }
  else { setUiSelected(null); setSelectedDrainNos([]); }
};
// store [] = 'use all drains in API' (backend convention)
// uiSelected null = 'show all checked in UI'
// Intentionally inverted to handle semantic mismatch
```

11.2 Cluster Area Warning Logic

```
const allClusters = multiClusterDistances
  ? multiClusterDistances.flatMap((g) => g.clusters)
  : clusterDistances ?? [];

const noneQualifies = allClusters.every((c) => (c.area_ha ?? 0) < landRequired);
// Shows orange warning if noneQualifies === true
```

11.3 Weighted Influence Normalization

```
// manualCategoryStore.ts -- buildWeightedSelections()
const totalInfluence = selections.reduce((s, r) => s + Number(r.Influence), 0);
return selections.map((r) => ({
  ...r,
  weight: (Number(r.Influence) / totalInfluence).toFixed(4),
}));
// Ensures weights always sum to 1.0
```

11.4 API Caching & Deduplication

Two Map caches in `manual_stpSuitabilityApi.ts` prevent duplicate simultaneous requests for identical payloads: **areaResultCache** (completed results) and **areaRequestCache** (in-flight Promises). The key is `JSON.stringify(payload)`. `findSuitabilityAreaClusterFresh()` clears both caches to force a fresh request.

12. Configuration & Constants

12.1 Technology Land per MLD

Technology	land (ha/MLD)
Trickling Filter	0.25
Activated Sludge Process	0.15
Extended Aeration	0.15
Sequential Batch Reactor	0.10
BIOFOR-F	0.08
Membrane Bioreactor	0.05
Constructed Wetland	0.30
Waste Stabilization Pond	0.40
Anaerobic Baffled Reactor	0.08
UASB + Constructed Wetland	0.15
Compact MBBR	0.06
Packaged Modular STP	0.05

12.2 STP Parameter Ranges

Q (MLD)	1 - 1000
Ce (Rs/kWh)	1 - 20
BOD (mg/L)	1 - 60,000
COD (mg/L)	1 - 100,000
Coliform (MPN/100mL)	1 - 1,000,000,000
Land (ha/MLD)	0 - 10
Capital (Cr/MLD)	0 - 100
O&M; (Rs/m3)	0 - 100
Reliability/Ease/Track	0 - 10 (score)
Num Clusters	1 - 10 (default 3)
Surface Radius	Default 5 km; adjustable

13. Error Handling & Edge Cases

13.1 User-Facing Errors

Scenario	Handling	Recovery
Constraint violations	Modal with violation list	Redraw or proceed via DSS
Land required > area	Amber warning, Find Area disabled	Redraw polygon or switch to DSS
All cluster areas too small	Orange warning below last cluster table	Larger polygon or different technology
API timeout (>10 min)	Error toast, loading cleared	Retry via New Analysis button
No drains in buffer	Empty drain dropdown	Expand surface radius or redraw
Zero categories (DSS)	Analyze button disabled	Select at least one condition layer
File format error	Toast notification with error message	Re-upload valid shapefile/KML

13.2 WebSocket / HTTP Polling

Primary	WebSocket on /tools/ws/operation/{taskId} -- instant push when task completes.
Fallback	HTTP polling every 2 seconds, max 300 attempts (10 minutes total).
Timeout	10 minutes hard limit; error shown and UI unblocked.
Deduplication	Duplicate requests for same payload share a single in-flight Promise.

13.3 Map Layer Persistence

All OL layers are stored in typed React refs (multiPolygonOutlineLayersRef, multiVillageLayersRef, multiResultLayersRef, polygonLabelLayerRef). Visibility toggle effects iterate these refs directly, ensuring correct toggling even when some entries lack a polygonLayer (e.g. drawn polygons without a GeoServer outline).

End of Documentation -- Manual STP Suitability Module | June 2026 | SLCR Deployment Team